SpendWise

Project Specification

Personal expense tracker | Version 1.0 | 31 October 2026

The first build

Build a small, complete web application that lets a person securely record daily expenses, organise them into categories and understand monthly spending in Australian dollars. SpendWise is a learning project for C#, ASP.NET Core, Blazor and relational data, with a usable release as the goal.

The MVP uses a single .NET 10 Blazor Web App with Interactive Server rendering, ASP.NET Core Identity, Entity Framework Core and SQLite. Browser interactions call server-side application services; a separate REST API is deferred. This keeps authentication, hosting and debugging within one application.

Delivery boundary

Plan for 35–45 focused hours over 7–10 working days, within one to two calendar weeks if that time is available. This assumes basic C# syntax, HTML and Git familiarity. At one hour per day, extend the calendar rather than adding shortcuts to security or testing.

MVP outcome	Release evidence
A private expense ledger	Two accounts cannot read or change each other’s records.
Accurate monthly spending	Expense list, total and category breakdown reconcile exactly.
A repeatable working release	Fresh setup, automated tests and a hosted smoke test pass.

How to use this specification

Implement the numbered requirements and milestones in order. The acceptance criteria define completion; stretch goals are optional only after the core release passes. Technical choices are project decisions unless identified as Microsoft guidance in the references.

Document map

2 Scope and user stories • 3 Technology • 4 Pages and interactions • 5 Architecture • 6 Data model • 7 Security • 8 Business rules • 9 Quality and testing • 10 Deployment • 11 Delivery and acceptance • 12 Structure and enhancements • 13 References

2 Scope and user stories

Target user and workflow

A student or working adult who wants a lightweight manual spending record. The primary flow is register → sign in → add an expense → filter the ledger → review the month → correct a record. There is one ordinary user role and no administration console.

ID	User story and required behaviour
FR01	As a user, I can register, sign in and sign out so my ledger stays private.
FR02	As a user, I can add an expense with amount, date and category, plus optional merchant and notes.
FR03	As a user, I can browse my expenses by month and optionally category, with newest dates first and 20 rows per page.
FR04	As a user, I can edit an expense or confirm its deletion so mistakes do not distort totals.
FR05	As a user, I can see the selected month's total, transaction count and spending by category.
FR06	As a user, I can use seeded categories and add, rename or archive my own categories.
FR07	As a user, I receive clear validation, empty, loading, reconnect and failure states without losing input during ordinary validation errors.
FR08	As a user, my saved records remain available after signing out, refreshing or an application restart.

Included in the MVP

Manual expense entry; AUD only; one account per person; personal categories; month navigation; optional category filter; dashboard totals and a simple category table. Default categories are Groceries, Dining, Transport, Housing, Utilities, Health, Entertainment and Other.

Explicit exclusions

No income, account balances, bank feeds, payments, tax calculations, shared households, receipt uploads, recurring expenses, exchange rates, AI categorisation, native mobile app or offline mode. Refunds and transfers are excluded; amounts are positive expenses only.

Release audience

The initial hosted release is a limited learning pilot using test data. Email confirmation and password recovery are deferred for that pilot and must be completed before an unrestricted public release with real personal data. The public release gate is separate from the MVP acceptance gate.

3 Technology and rationale

Layer	Choice	Reason and boundary
Language and runtime	C# 14 and .NET 10 LTS	One language across UI and server. Use the latest supported 10.0 patch; pin the installed SDK in global.json. [1]
Web framework	ASP.NET Core 10	Built-in hosting, dependency injection, configuration, logging and security middleware.
UI	Blazor Web App with Interactive Server	C# components and forms without a separate client application. Requires a live server connection; no offline support.
Authentication	ASP.NET Core Identity with cookies	Use the Individual Accounts template and generated account flows. Do not implement password hashing or token storage.
Persistence	EF Core 10 with SQLite provider	Migrations and LINQ teach relational access with no separate database service. Suitable for a small single-instance pilot.
Money storage	64-bit integer AUD cents	Exact aggregation and comparison. Use decimal for form input and convert once after validating two decimal places.
Styling	Template Bootstrap and small scoped CSS	Reuse available forms, spacing and responsive layout. Avoid adding a UI framework or chart library during the MVP.
Tests	xUnit and SQLite integration tests	Test rules and real relational behaviour. Browser smoke tests cover the complete workflow; bUnit and Playwright are optional.
Tools and delivery	Git, .NET CLI, VS Code or a compatible IDE	Cross-platform workflow. Use one repository, one web project and one test project; a basic CI build is sufficient.

Version discipline

Target net10.0. Keep Microsoft EF Core, Identity EF integration and dotnet-ef on compatible 10.x versions, and commit exact package versions plus a local tool manifest. Do not mix preview frameworks into this build. Choose .NET 10 instead of starting on .NET 8 or 9, whose support ends in November 2026. [1]

Alternatives deferred

A separate ASP.NET Core Web API plus Blazor WebAssembly adds HTTP contracts, a second execution environment and authentication decisions. SQL Server or PostgreSQL adds database hosting and provider migration work. These are useful later learning exercises, but do not improve the first release enough to justify the extra scope.

4 Pages and interactions

Page or route	Content and actions
/Account/Register and /Account/Login	Generated Identity screens. After successful authentication, navigate to the dashboard. Show generic sign-in errors; permit only local return URLs.
/	Authenticated dashboard. Month selector, AUD total, expense count, category totals sorted by spend, and Add expense action.
/expenses	Month and category filters; date, merchant or fallback label, category and amount; Edit and Delete actions; 20-row paging. Reset page number when filters change.
/expenses/new	Date defaults to today in Australia/Sydney. Amount, category, optional merchant and notes. Save and Cancel actions.
/expenses/{id}/edit	Same reusable form, prefilled from an owner-scoped query. A missing or other user's ID shows the same unavailable-record message.
/categories	Active categories, Add and Rename actions, archive confirmation and Show archived option with Restore. Referenced history remains visible.
Shared layout and account actions	Dashboard, Expenses and Categories navigation; signed-in account indicator and template sign-out POST. Responsive navigation on small screens.

Form and feedback behaviour

Use EditForm, DataAnnotationsValidator, a validation summary and field-level messages. Label required fields and display AUD beside the amount. Keep entered values on validation or database failure. Disable Save while an operation is running and show success only after the database commit.

Delete asks “Delete this A$24.50 expense?” and offers Cancel as the safe default. Deleting the last row on a page moves to the previous valid page. A failed deletion leaves the record visible with a retry message.

Empty and disconnected states

A new dashboard shows A$0.00 and “No expenses this month” with Add expense. A filter with no matches offers Clear category filter. Use the Blazor reconnect UI; disable actions while disconnected and offer refresh if reconnection fails. Unsaved input is not guaranteed after a full reload; say so before requesting one.

Responsive and accessible layout

At a 360-pixel viewport, forms use one column and ledger rows become readable cards or a contained table region. Maintain visible focus, keyboard access, associated labels, meaningful button names and sufficient colour contrast. Announce save/error status; never rely on colour alone.

5 Architecture and interaction design

One deployable application

Browser UI → Blazor Interactive Server circuit → application services → EF Core → SQLite. Identity account pages use their generated HTTP request/response flows. All database access and business rules execute on the server; the browser never receives a connection string.

Components own presentation, form models and loading state. Services own validation, user ownership, queries and writes. EF Core owns mappings and migrations. Keep these as folders within the web project; a generic repository, CQRS, message bus and multiple service projects are unnecessary.

Service contracts for the MVP

Operation	Contract and result
ListExpensesAsync(filter)	Month, optional category and page → paged ExpenseRow DTOs plus matching count.
GetExpenseAsync(id)	Owner-scoped ID → editable DTO or NotFound.
CreateExpenseAsync(input)	Validated form DTO → new ID. Owner and timestamps assigned on the server.
UpdateExpenseAsync(id, input)	Owner-scoped update → Success, ValidationFailure or NotFound.
DeleteExpenseAsync(id)	Owner-scoped delete → Success or NotFound; confirmation is in the UI.
GetMonthlySummaryAsync(month)	Month → total cents, count and per-category totals for the current user.
Category service operations	List, create, rename, archive and restore categories; enforce ownership and unique names.

The services obtain the authenticated user ID from a server-side current-user abstraction backed by AuthenticationStateProvider. Do not accept an owner ID from a form, URL or browser payload. Use DTOs so callers cannot assign protected entity fields.

Database lifetime and errors

Create and dispose a fresh DbContext per application service operation using IDbContextFactory. Do not retain it in an interactive component or share it across parallel calls. Keep template Identity registration intact and make context-factory option lifetimes compatible; verify registration and sign-in after wiring the factory. [2]

Use async EF calls and AsNoTracking for reads. Apply ownership and filters before paging. Return predictable validation/not-found outcomes; log unexpected failures with a correlation ID and show a generic error. Keep an operation busy flag to prevent repeated submissions.

API decision

No public expense REST endpoints are required for the MVP. Blazor events invoke these services in process. A future API can expose GET/POST /api/expenses, GET/PUT/DELETE /api/expenses/{id}, GET /api/summary and category routes, reusing the same ownership rules. API authentication and HTTP status mapping belong to that later phase.

6 Database schema and relationships

Use one ApplicationDbContext derived from IdentityDbContext<ApplicationUser>. Standard Identity tables manage users, credentials, claims and tokens; do not recreate them. SQLite is the development and pilot database so tests match deployment behaviour.

Entity and field	Type and constraints
ApplicationUser	Inherits IdentityUser; string Id primary key. Use template email and normalised email fields; configure unique email. No custom profile fields required.
Category.Id and UserId	Id: int primary key. UserId: required string foreign key to ApplicationUser.Id.
Category.Name and NormalizedName	Required strings, 40 characters maximum; trim Name and use invariant upper case for NormalizedName.
Category.IsArchived	Required bool, default false.
Expense.Id and UserId	Id: int primary key. UserId: required string foreign key to ApplicationUser.Id.
Expense.CategoryId	Required int; owner-matching composite foreign key described below.
Expense.AmountCents	Required long mapped to INTEGER; check between 1 and 99,999,999 inclusive.
Expense.ExpenseDate	Required DateOnly, mapped by the provider to TEXT. An expense calendar date, not an instant.
Expense.Merchant and Notes	Nullable strings; trimmed; merchant maximum 100 characters and notes maximum 500. Empty strings become null.
Expense.CreatedAtUtc and UpdatedAtUtc	Required DateTime values stored in UTC; created set once, updated set on each successful write.

Relationships and integrity

ApplicationUser 1 → many Category; ApplicationUser 1 → many Expense; Category 1 → many Expense. Create an alternate key on Category (Id, UserId), then map Expense (CategoryId, UserId) to it. This prevents an expense from referring to another user’s category even if a service check is missed.

Create a unique index on Category (UserId, NormalizedName), including archived categories. Index Expense (UserId, ExpenseDate, Id) for the ledger and add (UserId, CategoryId, ExpenseDate) for category filtering. Enforce required foreign keys. Use restrictive delete behaviour; account deletion and hard category deletion are not exposed in the MVP.

Migrations and initial categories

Commit EF migrations. After registration, initialise the eight categories once for that user in a transaction. Make initialisation idempotent using the unique name index and retry-safe existence checks. Never seed a public shared demo password. SQLite does not enforce declared string lengths, so validate lengths in services as well as forms. [4]

7 Authentication and security

Identity and session handling

Start from the .NET 10 Blazor Web App template with Individual Accounts. Preserve generated account components, endpoint handling and authentication-state revalidation. Login and logout need normal HTTP responses to manage cookies; do not attempt cookie sign-in inside an established interactive circuit. [3]

Use Identity password hashing. Configure unique email, minimum password length of 12, and permit password-manager paste. Enable lockout on failed sign-in attempts: five failures trigger a 15-minute lockout. Use HTTPS with Secure and HttpOnly authentication cookies, SameSite=Lax and a 60-minute sliding session expiry for the pilot. Verify actual deployed cookie settings.

Authorisation is enforced in services

- Require authentication on all application routes except registration and login. Redirect unauthenticated visitors to sign-in.
- Every expense/category read, summary, update and delete includes the current user ID in its database predicate. A hidden UI button is not an access control.
- Resolve a selected category by both ID and owner. Reject another user's category even if the submitted ID exists.
- Treat another user's record as unavailable without confirming its existence. Apply the same policy to category actions and dashboard aggregates.

Input and infrastructure protection

Validate again on the server even after form validation. Use EF parameterised queries and normal Razor encoding; render notes as text, never raw HTML. Retain template antiforgery protection for cookie-authenticated HTTP forms and logout. Do not create state-changing GET endpoints.

Store local secrets with .NET user-secrets and deployed secrets in the hosting secret store. Keep database files, backups, data-protection keys and secrets outside wwwroot and out of Git. Restrict file permissions to the service account. Persist and protect data-protection keys across restarts.

Do not log passwords, cookies, notes, merchant names or expense payloads. Log operation, result, timing and correlation ID where useful. Disable detailed exception pages outside Development; configure HTTPS redirection and production HSTS.

Pilot and public release gates

For the limited test-data pilot, confirmation may be disabled explicitly and email recovery links must not pretend to work. Before public registration with real data, configure an email sender, enable confirmation and reset flows, remove any development confirmation-link exposure, add abuse controls, and document account deletion and retention. Do not treat the pilot as a production financial service.

8 Business rules and worked example

Rule	Required behaviour
BR01 Amount	Accept A$0.01 through A$999,999.99. Reject zero, negative amounts, more than two decimal places and invalid input. Do not silently round.
BR02 Conversion	Parse with en-AU culture into decimal. After validation, convert amount × 100 to long using a checked conversion. Format cents ÷ 100m as AUD; never use float or double for money.
BR03 Dates	Allow 1 January 2000 through today in Australia/Sydney inclusive. Use an injectable clock/time-zone helper. No future expenses. Store date-only values without UTC conversion.
BR04 Month boundaries	Include dates from the first day of the selected month up to, but not including, the first day of the next month. Allow month selection within the supported date range.
BR05 Categories	Names are trimmed and unique per user ignoring case, 1–40 characters. Archived names remain reserved; restore or rename instead of creating a duplicate.
BR06 Archived history	An archived category stays on existing records and totals but is unavailable for new assignments. An existing expense may retain its archived category when other fields are edited.
BR07 Summaries	Sum all owned expenses in the selected month. Dashboard does not inherit the ledger category filter. Category totals must add to the dashboard total; no expenses means zero.
BR08 Edits and duplicates	A successful edit refreshes affected lists and totals. Identical expenses are allowed because repeated purchases are valid. Disable repeated Save while in progress.
BR09 Concurrent changes	MVP uses last successful write wins for two tabs editing the same expense. If already deleted, return NotFound. Conflict detection and retry deduplication are later enhancements.

Acceptance fixture in Australian dollars

For user A in October 2026, record groceries A$82.40 on 3 October, transport A$12.00 on 4 October, and dining A$24.50 on 5 October. Add A$9.00 on 30 September and A$100.00 for user B in October.

October for user A must show A$118.90 across three expenses. Category totals are A$82.40, A$12.00 and A$24.50. The September expense and user B’s expense are excluded. Filtering the ledger to Dining shows A$24.50; the dashboard remains A$118.90. Editing dining to A$20.00 changes October to A$114.40; deleting transport then gives A$102.40 across two expenses.

9 Quality requirements and testing

ID	Target and verification
NFR01 Accuracy	Exact cent arithmetic and correct calendar boundaries. Unit tests cover minimum, maximum, invalid precision, leap day and month/year rollover.
NFR02 Isolation	No cross-user data access through any application service. Automated two-user integration tests are a release blocker.
NFR03 Performance	With 5,000 expenses for one user, warm list and summary service calls each complete within 500 ms locally; visible operations within 2 seconds on the pilot host under normal network conditions. Record environment and 20-run p95; these are targets, not measured claims.
NFR04 Usability	Add a typical expense in under 30 seconds after sign-in. Verify with one first-time tester; check 360 px and desktop layouts, keyboard use and error messages.
NFR05 Reliability	Committed data survives restart; failure never displays success. Demonstrate backup restore. Unsaved browser state and offline operation are not guaranteed.
NFR06 Maintainability	A clean checkout can build, test and run using the README. No committed secrets, unhandled build warnings from application code or undocumented setup steps.

Automated tests to implement first

Unit tests: money parsing/conversion; date boundaries using a fixed clock; month filtering inputs; category normalisation; limits on merchant and notes. Include A$0.01, A$999,999.99, A$0.00, -A$1.00 and 1.005 as boundary cases.

Integration tests: use a temporary SQLite database with real migrations, not EF InMemory. Verify create/update/delete persistence, foreign keys, duplicate category rejection, archive behaviour, filtered paging and the worked totals. Test user B’s IDs against every read and mutation service; verify both the failure outcome and unchanged database state.

Manual end to end release pass

Register two users; sign in and out; add invalid then valid expenses; refresh; navigate months; edit and delete; archive and restore categories. Exercise a stale deleted record and an attempted foreign-user ID. Confirm empty states, disabled Save, keyboard focus, mobile layout and reconnect behaviour.

Restart the host and sign in again to confirm persistence. Restore a backup to a separate temporary path and compare row counts and the known fixture totals. Record results in a short test checklist. Optional bUnit tests cover form errors; optional Playwright automation covers the happy path after the MVP is stable.

10 Deployment and operations

Recommended pilot deployment

Use one Linux host or container instance with .NET 10, HTTPS, WebSocket support and an attached persistent local disk. Run the SQLite file on that disk, not in the application image or an ephemeral publish directory. Choose a region near the test users. A static website host cannot run this application. [5]

Keep the pilot at one application replica and a few testers. SQLite and server-side circuits make multi-instance hosting a separate design task. For Azure App Service or another managed platform, verify WebSockets, supported runtime, storage persistence and any session-affinity requirements before choosing a plan. Do not assume a free tier has these properties. [5]

Release procedure

1. Run restore, Release build and all tests from a clean checkout. Commit migrations and the tested package versions.

2. Publish the web project in Release mode. Configure Production environment, database path and protected data-protection key storage. Use the host secret store for any credentials.

3. Back up the current database and record the application version. Stop the application before schema changes in this single-instance pilot.

4. Apply committed EF migrations as a controlled release step. Avoid concurrent startup migrations. Start the service under a restricted account behind HTTPS.

5. Run the two-user smoke test over the deployed URL. Confirm cookies, database persistence, reconnect behaviour and logs without sensitive payloads.

6. On failure, stop the service and restore the prior application plus its compatible database backup. Do not assume a destructive migration can be reversed safely.

Backups and recovery

For the pilot, stop the application and copy the SQLite database to a protected backup location before each release and at least daily when test data changes. A live-file copy is unsafe with active writes or journal files; use SQLite’s backup mechanism if downtime-free backup is later needed. Keep seven daily backups and verify one restore before release.

Pilot recovery targets are at most 24 hours of data loss and restoration within one hour; validate the procedure rather than promising uptime. Protect backups like the original database and never publish them as portfolio assets.

Configuration and documentation

README must cover SDK version, local setup, database creation, test commands, deployment assumptions, backup/restore, pilot limitations and screenshots with invented data. Log startup failures and unexpected operation errors. Add a simple health endpoint returning status only; do not expose connection details or user information.

11 Milestones and acceptance criteria

Stage	Hours	Deliverable and exit condition
Days 1–2 Foundation	6–8	Create template and Git repository; confirm Identity login; add entities, factory and first migration. Two users have independent seeded categories.
Days 3–4 Ledger	8–10	Expense create/list/edit/delete with ownership, validation and paging. Refresh preserves records; negative ownership tests pass.
Days 5–6 Insights	6–8	Month/category filters, dashboard and category archive/restore. Worked example reconciles exactly.
Days 7–8 Verification	7–9	Unit and integration tests, responsive states, keyboard checks and first-time tester feedback. Fix release-blocking issues.
Days 9–10 Release	8–10	Deploy one instance, test restart and restore, finish README and demo. Includes setup/debugging contingency.

Total estimate: 35–45 focused hours. A one-week delivery needs near full-time availability; two weeks is the safer learning plan. If delayed, remove visual polish and optional automation first, add no stretch features, and keep security, accuracy and persistence checks intact.

Release acceptance checklist

AC01 — FR01 and NFR02: two users can register and sign in; anonymous app access is blocked; foreign expense and category IDs never disclose or mutate another user's records.

AC02 — FR02, FR04 and BR01–03: valid expenses save; all boundary errors are actionable; edit and confirmed delete persist and update totals.

AC03 — FR03 and BR04: month/category filters and 20-row paging return only matching owned records in ExpenseDate descending, then Id descending order.

AC04 — FR05 and BR07: the October fixture produces A$118.90 and the specified edit/delete totals with exact category reconciliation.

AC05 — FR06 and BR05–06: categories seed once; duplicate names fail; archive hides new assignments but keeps history; restore works.

AC06 — FR07 and NFR04: small-screen and keyboard workflows pass; empty, loading, error and reconnect states are understandable.

AC07 — FR08 and NFR05: a saved expense survives restart and a backup restores the known totals.

AC08 — NFR03 and NFR06: performance measurements are recorded; clean setup, build and tests pass; deployed smoke test and README are complete.

Definition of done: all eight criteria have recorded evidence, no known cross-user access or calculation defects remain, and a short demo shows the complete workflow. Public registration with real data additionally requires the security gate on page 7.

12 Folder structure and future enhancements

Suggested solution structure

```
SpendWise/
  SpendWise.slnx
  global.json
  .config/dotnet-tools.json
  src/SpendWise.Web/
    Components/
      Account/            generated Identity flows
      Layout/             navigation and shell
      Pages/              Dashboard and Expenses and Categories
      Shared/             ExpenseForm and status components
    Data/
      ApplicationDbContext.cs
      Configurations/     entity mappings and indexes
      Migrations/
    Models/               ApplicationUser and Expense and Category
    Contracts/            input and result DTOs
    Services/             expense and category and summary services
    Security/             current-user abstraction
    Validation/           money and date rules
    wwwroot/              CSS and public static assets only
    Program.cs
    appsettings.json      non-secret defaults
  tests/SpendWise.Tests/
    Unit/
    Integration/
  docs/                   setup notes and acceptance evidence
  .github/workflows/ci.yml
  README.md
  .gitignore
```

Keep generated Identity code recognisable. Ignore local SQLite files, backups, user secrets, bin and obj. The solution may use .sln instead of .slnx if the selected tooling requires it.

Stretch goals in recommended order

1. Monthly budget: one AUD limit per user per month, with remaining/overspent values and boundary tests. 2. CSV export: export only owned, filtered records and mitigate spreadsheet formula injection. 3. Charts: visualise the existing summary data with an accessible table alternative.

4. Concurrency protection: add an application-managed version token and conflict message, plus request deduplication where needed. 5. Public release readiness: verified email, recovery, deletion/retention policy and abuse controls. 6. REST API learning phase: authenticated endpoints, DTO contracts and integration tests, then consider a separate client.

Later product ideas

Recurring expenses, receipt images, income, multiple currencies and bank connections each need fresh requirements and security review. Validate which feature a real user needs before expanding. Add one enhancement at a time and retain the working MVP as a tagged release.

13 References and implementation notes

Official technical references checked on 31 October 2026. These sources support framework and hosting choices; the scope, hour estimates, limits and performance targets in this specification are project decisions.

[1] .NET support policy

Use a supported .NET 10 LTS patch and recheck support status when starting the build.

https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core

[2] Blazor with Entity Framework Core

Guidance on context lifetime and one context per operation for interactive server components.

https://learn.microsoft.com/en-us/aspnet/core/blazor/blazor-ef-core?view=aspnetcore-10.0

[3] ASP.NET Core Identity documentation source

Documents the generated Identity components in a Blazor Web App with Individual Accounts. Preserve the current template's account flow.

https://github.com/dotnet/AspNetCore.Docs/blob/main/aspnetcore/security/authentication/identity.md

[4] SQLite EF Core provider limitations

Provider-specific schema and migration limitations. Integer cents are a deliberate project choice for exact, simple money handling.

https://learn.microsoft.com/en-us/ef/core/providers/sqlite/limitations

[5] Host and deploy server-side Blazor

Server connection and hosting guidance, including WebSockets and platform-specific affinity considerations.

https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/blazor/server

First implementation checkpoint

Create the Individual Accounts template, run it, register two test users, and prove login and logout before adding expense features. Then add the data model and migrations, and complete one vertical slice: add an expense, save it and show it in the ledger. This exposes integration problems early while the application is still small.